

Design WhatsApp

⚡ Real-time Updates

By Stefan Mai · Published Jul 17, 2024 ·  medium · Asked at:  +10



Try This Problem Yourself

Practice with guided hints and real-time feedback

 Start Practice



Watch Video Walkthrough

Watch the author walk through the problem step-by-step

 Watch Now

Understanding the Problem

What is Whatsapp?

Whatsapp is a messaging service that allows users to send and receive encrypted messages and calls from their phones and computers. Whatsapp was famously originally built on Erlang (no longer!) and renowned for handling high scale with limited engineering and infrastructure outlay.

Functional Requirements

Apps like WhatsApp and Messenger have tons of features, but your interviewer doesn't want you to cover them all. The most obvious capabilities are almost definitely in-scope but it's good to ask your interviewer if they want you to move beyond. Spending too much time in requirements will make it harder for you to give detail in the rest of the interview, so we won't dawdle too long here!

Core Requirements

1. Users should be able to start group chats with multiple participants (limit 100).
2. Users should be able to send/receive messages.
3. Users should be able to receive messages sent while they are not online (up to 30 days).
4. Users should be able to send/receive media in their messages.



That third requirement isn't obvious to everyone (but it's interesting to design) and if I'm your interviewer I'll probably guide you to it.

Below the line (out of scope)

1. Audio/Video calling.
2. Interactions with businesses.
3. Registration and profile management.

Non-Functional Requirements

Core Requirements

1. Messages should be delivered to available users with low latency, < 500ms.
2. We should guarantee deliverability of messages - they should make their way to users.
3. The system should be able to handle billions of users with high throughput (we'll estimate later).
4. Messages should be stored on centralized servers no longer than necessary.
5. The system should be resilient against failures of individual components.

Below the line (out of scope)

1. Exhaustive treatment of security concerns.
2. Spam and scraping prevention systems.



Adding features that are out of scope is a "nice to have". It shows product thinking and gives your interviewer a chance to help you reprioritize based on what they want to see in the interview. That said, it's very much a nice to have. If additional features are not coming to you quickly (or you've already burned some time), don't waste your time and move on. It's easy to use precious time defining features that are out of scope, which provides negligible value for a hiring decision.

Functional

Start group chats (limit 100 users)
Send-receive messages
Send/receive media
Access messages after they've been offline

Non-Functional

Delivered with low-latency, < 500ms
Guarantee delivery of messages
Billions of users, high throughput
Messages not stored unnecessarily
Fault-tolerant



Requirements

The Set Up

Planning the Approach

Before you move on to designing the system, it's important to start by taking a moment to plan your strategy for the session. For this problem, we might first recognize that 1:1 messages are simply a special case of larger chats (with 2 participants), so we'll solve for that general case.

After this, we should be able to start our design by walking through our core requirements and solving them as simply as possible. This will get us started with a system that is probably slow and not scalable, but a good starting point for us to optimize in the deep dives.

In our deep dives we'll address scaling, optimizations, and any additional features/functionality the interviewer might want to throw on the fire.

Defining the Core Entities

In the core entities section, we'll think through the main "nouns" of our system. The intent here is to give us the right language to reason through the problem and set the stage for our API and data model.

language to reason through the problem and set the stage for our API and data model.



Interviewers aren't evaluating you on what you list for core entities, they're an intermediate step to help you reason through the problem. That doesn't mean they don't matter though! Getting the entities wrong is a great way to start building on a broken foundation - so spend a few moments to get them right and keep moving.

We can walk through our functional requirements to get an idea of what the core entities are. We need:

- Users
- Chats (2-100 users)
- Messages
- Clients (a user might have multiple devices)

We'll use this language to reason through the problem.

API or System Interface

Next, we'll want to think through the API of our system. Unlike a lot of other products where a REST API is probably appropriate, for a chat app, we're going to have high-frequency updates being both sent and received. This is a perfect use case for a bi-directional socket connection!

⚡ Pattern: Real-time Updates

WebSocket connections and real-time messaging demonstrate the broader **real-time updates pattern** used across many distributed systems. Whether it's chat messages, live dashboards, collaborative editing, or gaming, the same principles apply: persistent connections for low latency, pub/sub for scaling across servers, and careful state management for reliability.

[Learn This Pattern](#)

For this interview, we'll just use websockets although a simple TLS connection would do. The idea will be that users will open the app and connect to the server, opening this socket which will be used to send and receive commands which represent our API.

Just knowing that we have a websocket connection is useful, but we'll need to know what commands we want to exchange on the socket.

First, let's be able to create a chat.

```
// -> createChat
{
  "participants": [],
  "name": ""
} -> {
  "chatId": ""
}
```

Now we should be able to send messages on the chat.

```
// -> sendMessage
```

```
{
  "chatId": "",
  "message": "",
  "attachments": []
} -> "SUCCESS" | "FAILURE"
```

We need a way to create attachments (note: I'm going to amend this later in the writeup).

```
// -> createAttachment
{
  "body": ...,
  "hash":
} -> {
  "attachmentId": ""
}
```

And we need a way to add/remove users to the chat.

```
// -> modifyChatParticipants
{
  "chatId": "",
  "userId": "",
  "operation": "ADD" | "REMOVE"
} -> "SUCCESS" | "FAILURE"
```

Each of these commands will have a parallel commands that is sent to other clients. When the command has been received by clients, they'll send an `ack` command back to the server letting it know the command has been received (and it doesn't have to be sent again)!



The message receipt acknowledgement is a bit non-obvious but crucial to making sure we don't lose messages. By forcing clients to ack, we can know for certain that the message has been delivered all the way to the client.

When a chat is created or updated ...

```
// <- chatUpdate
{
  "chatId": "",
  "participants": [],
} -> "RECEIVED"
```

When a message is received ...

```
// <- newMessage
{
  "chatId": "",
  "userId": "",
  "message": "",
  "attachments": []
} -> "RECEIVED"
```

Etc ...

Note that enumerating all of these APIs can take time! In the actual interview, I might shortcut by only writing the command names and not the full API. It's also usually a good idea to summarize the API initially before you build out the high-level design in case things need to change. "I'll come back to this as I learn more" is completely acceptable!

Our whiteboard might look like this:

Commands Sent

- * createChat
- * sendMessage
- * createAttachment
- * modifyParticipants

Commands Received

- * newMessage
- * chatUpdate



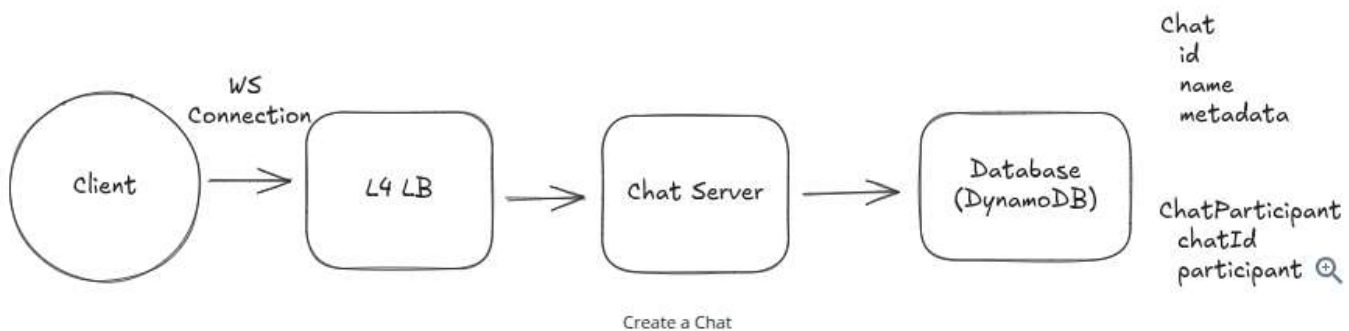
Commands Exchanged

Now that we have a base to work with let's figure out how we can make them real while we satisfy our requirements.

High-Level Design

1) Users should be able to start group chats with multiple participants (limit 100)

For our first requirement, we need a way for a user to create a chat. We'll start with a simple service behind an L4 load balancer (to support Websockets!) which can write `Chat` metadata to a database. Let's use `DynamoDB` for fast key/value performance and scalability here, although we have lots of other options.



The steps here are:

1. User connects to the service and sends a `createChat` message.
2. The service, inside a transaction, creates a `Chat` record in the database and creates a `ChatParticipant` record for each user in the chat.
3. The service returns the `chatId` to the user.

On the chat table, we'll usually just want to look up the details by the chat's ID. Having a simple primary key on the

chat `id` is good enough for this.

For the `ChatParticipant` table, we'll want to be able to (1) look up all participants for a given chat and (2) look up all chats for a given user.

1. We can do this with a composite primary key on the `chatId` and `participantId` fields. A range lookup on the `chatId` will give us all participants for a given chat.
2. We'll need a **Global Secondary Index (GSI)** with `participantId` as the partition key and `chatId` as the sort key. This will allow us to efficiently query all chats for a given user. The GSI will automatically be kept in sync with the base table by DynamoDB.

Great! We got some chats. How about messages?

2) Users should be able to send/receive messages.

To allow users to send/receive messages, we're going to need to start taking advantage of the websocket connection that we established. To keep things simple while we get off the ground, let's assume we have a single host for our Chat Server.

This is obviously a terrible solution for scale (and you might say so to your interviewer to keep them from itching), but it's a good starting point that will allow us to incrementally solve those problems as we go.



For infrastructure-style interviews, I highly recommend reasoning about a solution on a single host first. Oftentimes the path to scale is straightforward from there. On the other hand if you solve scale first without thinking about how the actual mechanics of your solution work underneath, you're likely to back yourself into a corner.

When users make Websocket connections to our Chat Server, we'll want to keep track of their connection with a simple hash map which will map a user id to a websocket connection. This way we know which users are connected and can send them messages.

To send a message:

1. User sends a `sendMessage` message to the Chat Server.
2. The Chat Server looks up all participants in the chat via the `ChatParticipant` table.
3. The Chat Server looks up the websocket connection for each participant in its internal hash table and sends the message via each connection.

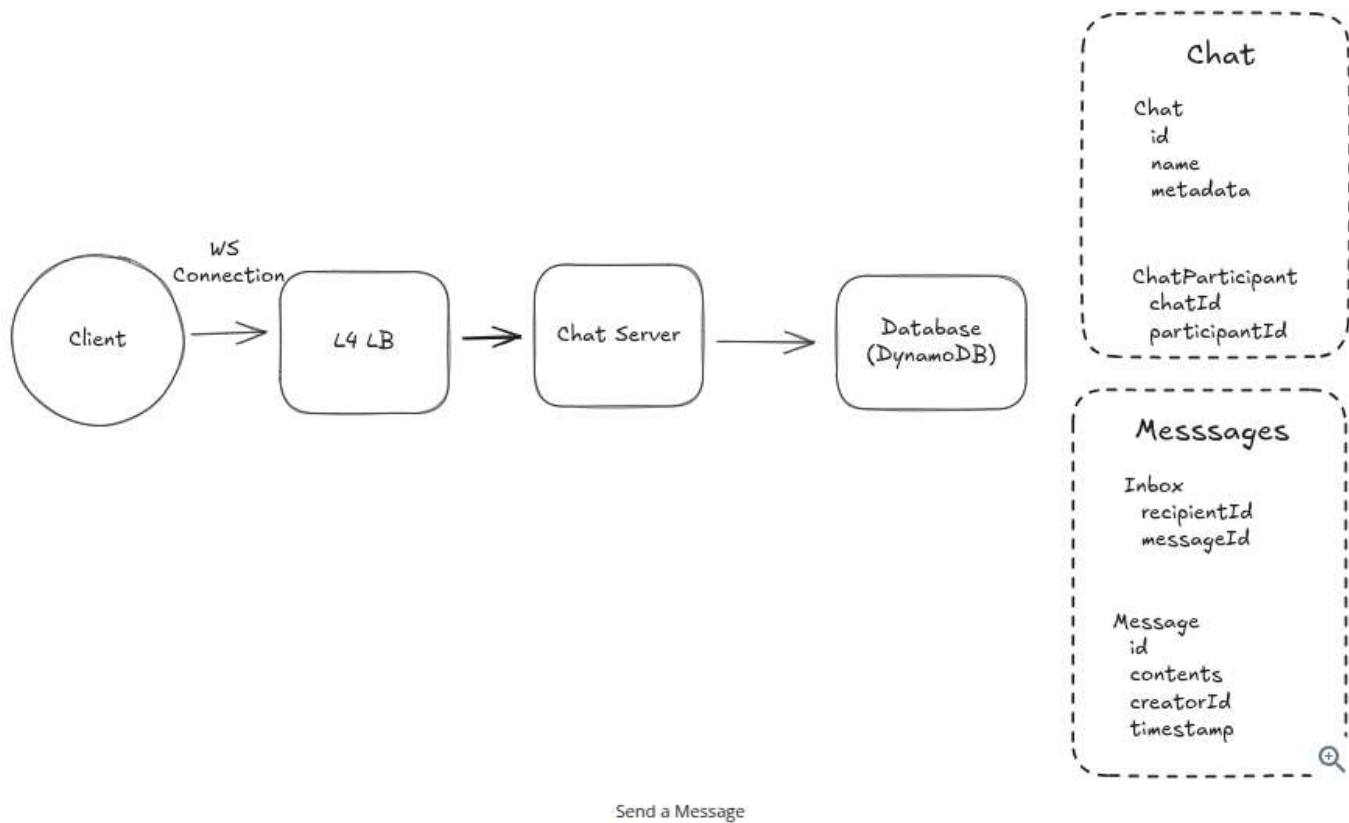
We're making some really strong assumptions here! We're assuming all users are online, connected to the same Chat Server, and that we have a websocket connection for each of them. But under those conditions we're moving, so let's keep going.

3) Users should be able to receive messages sent while they are not online (up to 30 days).

With our next requirement, we're forced to undo some of those assumptions. We're going to need to start storing messages in our database so that we can deliver them to users even when they're offline. We'll take this as an opportunity to add some robustness to our system.

Let's keep an "Inbox" for each user which will contain all undelivered messages. When messages are sent, we'll write

them to the inbox of each recipient user. If they're already online, we can go ahead and try to deliver the message immediately. If they're not online, we'll store the message and wait for them to come back later.



So, to send a message:

1. User sends a `sendMessage` message to the Chat Server.
2. The Chat Server looks up all participants in the chat via the `ChatParticipant` table.
3. The Chat Server creates a transaction which both (a) writes the message to our `Message` table and (b) creates an entry in our `Inbox` table for each recipient.
4. The Chat Server returns a `SUCCESS` or `FAILURE` to the user with the final message id.
5. The Chat Server looks up the websocket connection for each participant and attempts to deliver the message to each of them via `newMessage`.
6. (For connected clients) Upon receipt, the client will send an `ack` message to the Chat Server to indicate they've received the message. The Chat Server will then delete the message from the `Inbox` table.

For clients who aren't connected, we'll keep the messages in the `Inbox` table. Once the client connects to our service later, we'll:

1. Look up the user's `Inbox` and find any undelivered message IDs.
2. For each message ID, look up the message in the `Message` table.
3. Write those messages to the client's connection via the `newMessage` message.
4. Upon receipt, the client will send an `ack` message to the Chat Server to indicate they've received the message.
5. The Chat Server will then delete the message from the `Inbox` table.

Finally, we'll need to periodically clean up the old messages in the `Inbox` and messages tables. We can do this with a simple cron job which will delete messages older than 30 days.

Great! We knocked out some of the durability issues of our initial solution and enabled offline delivery. Our solution still doesn't scale and we've got a lot more work to do, so let's keep moving.

4) Users should be able to send/receive media in their messages.

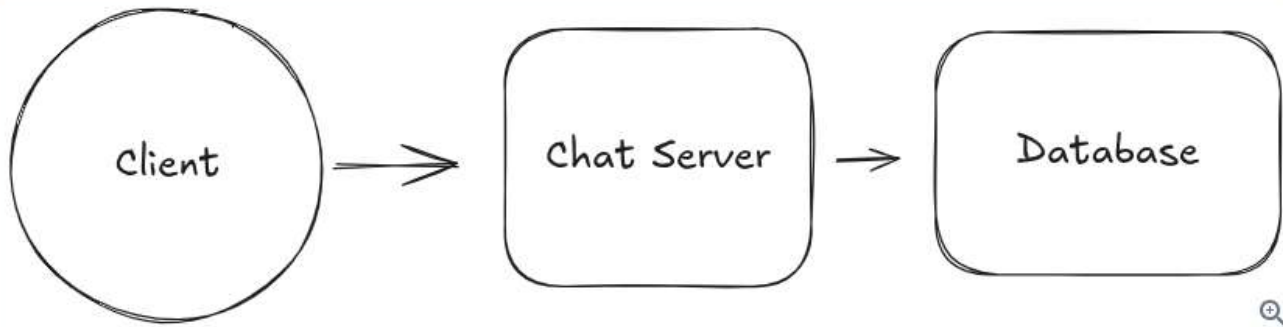
Our final requirement is that users should be able to send/receive media in their messages.

Users sending and receiving media is annoying. It's bandwidth- and storage- intensive. While we could potentially do this with our Chat Server and database, it's better to use purpose-built technologies for this. This is in fact how Whatsapp actually works: attachments are uploaded via a separate HTTP service.

Bad Solution: Keep attachments in DB

Approach

The worst approach is to have the Chat Server accept the attachment media over our websocket connection and save it in our database. Then we'll need to add an additional message type for users to retrieve attachments.



Attachments in DB

Challenges

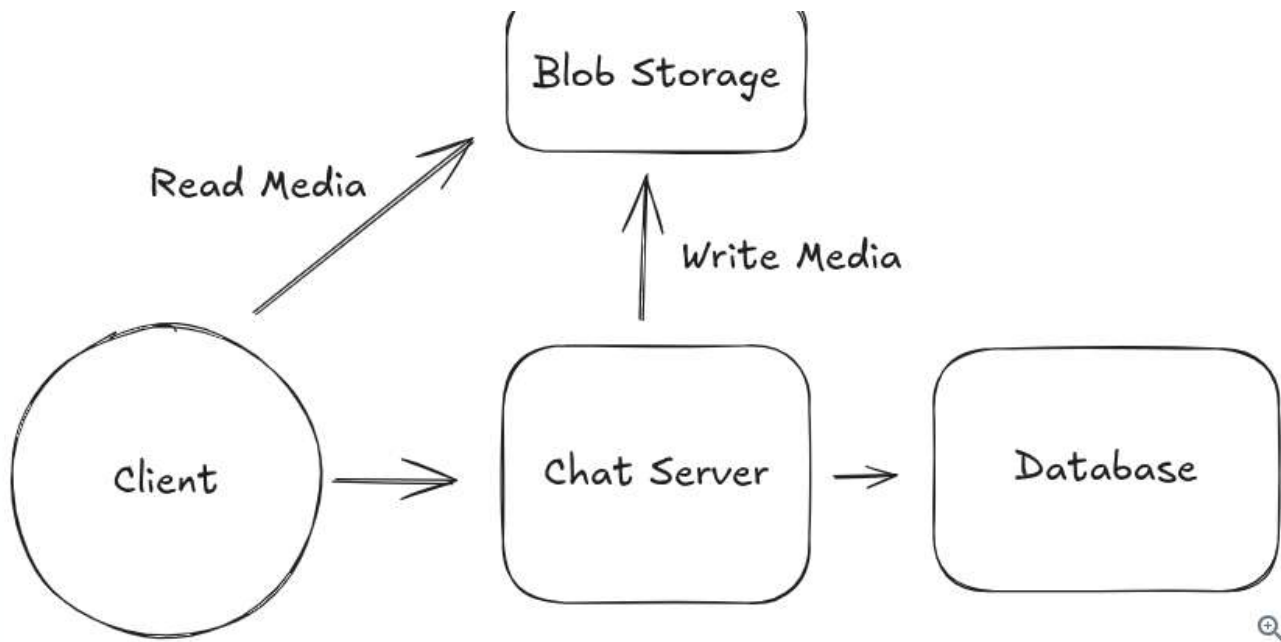
This is not a good solution. First, most databases (including DynamoDB) aren't optimized for handling large binary blobs. Second, we're crippling the bandwidth available to our Chat Servers by occupying them with comparatively dumb storage and retrieval. There are better solutions!

Good Solution: Send attachments via chat server

Approach

A straightforward approach is to have the Chat Server accept the attachment media, then push it off to blob storage with a TTL of 30 days (remember we don't need to keep messages forever!).

Users who want to retrieve a particular attachment can then query the blob storage directly (via a pre-signed URL for authorization). Ideally, we'd find a way to expire the media once it had been received by all recipients. While we could put a CDN in front of our blob storage, since we're capped at 100 participants the cache benefits are going to be relatively small.



Use Blob Storage"

Challenges

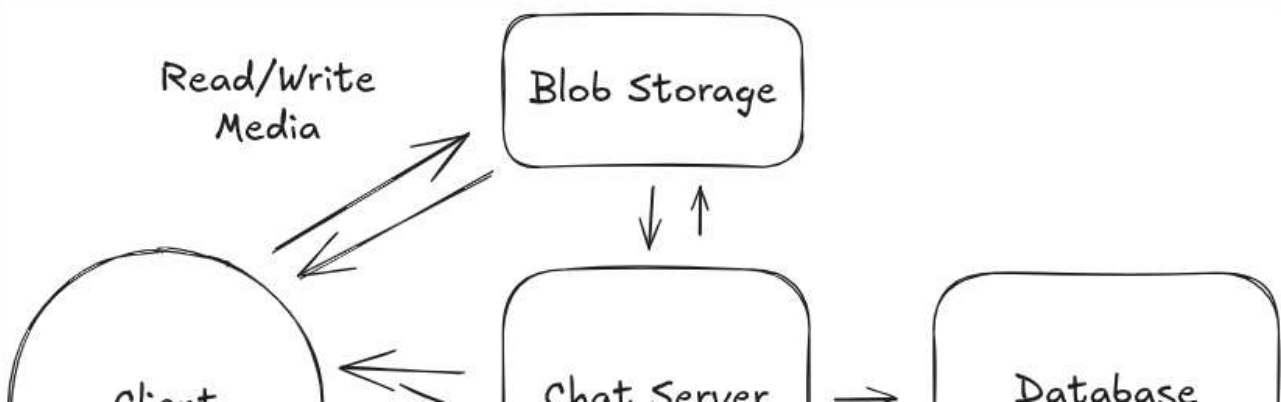
Unfortunately, our Chat Servers still have to handle the incoming media and forward it to the blob storage (a wasted step). Expiring attachments once they've been downloaded by all recipients isn't handled. Managing encryption and security will require extra steps.

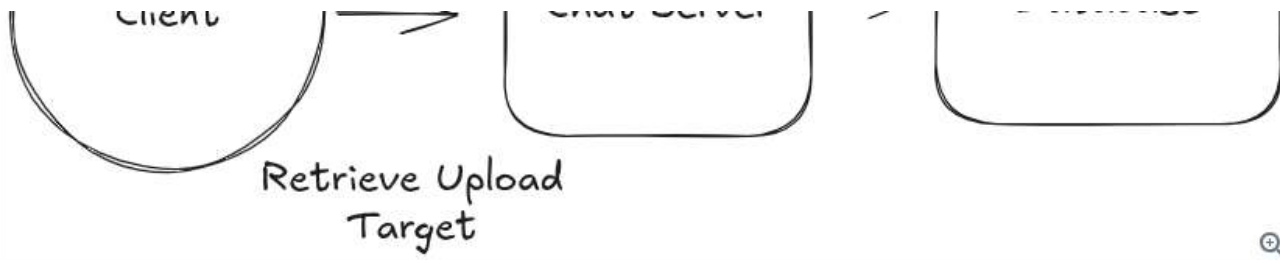
Great Solution: Manage attachments separately

Approach

An ideal approach is that we give our users permission (e.g. via pre-signed URLs) to upload directly to the blob storage. As an example, they might send a `getAttachmentTarget` message to the Chat Server which returns a pre-signed URL. Once uploaded, the user will have a URL for the attachment which they can send to the Chat Server as an opaque URL.

Then, our solution works much like the "Good" solution. Users who want to retrieve a particular attachment can then query the blob storage directly (via a pre-signed URL for authorization). Ideally, we'd find a way to expire the media once it had been received by all recipients. While we could put a CDN in front of our blob storage, since we're capped at 100 participants the cache benefits are going to be relatively small.





Challenges

Expiring attachments once they've been downloaded by all recipients isn't handled. Managing encryption and security will require extra steps.

Ok awesome, so we have a system which has real-time delivery of messages, persistence to handle offline use-cases, and attachments.

Potential Deep Dives

With the core functional requirements met, it's time to dig into the non-functional requirements via deep dives and solve some of the issues we've earmarked to this point. This includes solving obvious scalability issues as well as auxiliary questions which demonstrate your command of system design.



The degree to which a candidate should proactively lead the deep dives is a function of their seniority. In this problem, all levels should be quick to point out that my single-host solution isn't going to scale. But beyond these bottlenecks, it's reasonable in a mid-level interview for the interviewer to drive the majority of the deep dives. However, in senior and staff+ interviews, the level of agency and ownership expected of the candidate increases. They should be able to proactively look around corners and identify potential issues with their design, proposing solutions to address them.

1) How can we handle billions of simultaneous users?

Our single-host system is convenient but unrealistic. Serving billions of users via a single machine isn't possible and it would make deployments and failures a nightmare. So what can we do? The obvious answer is to try to scale out the number of Chat Servers we have.

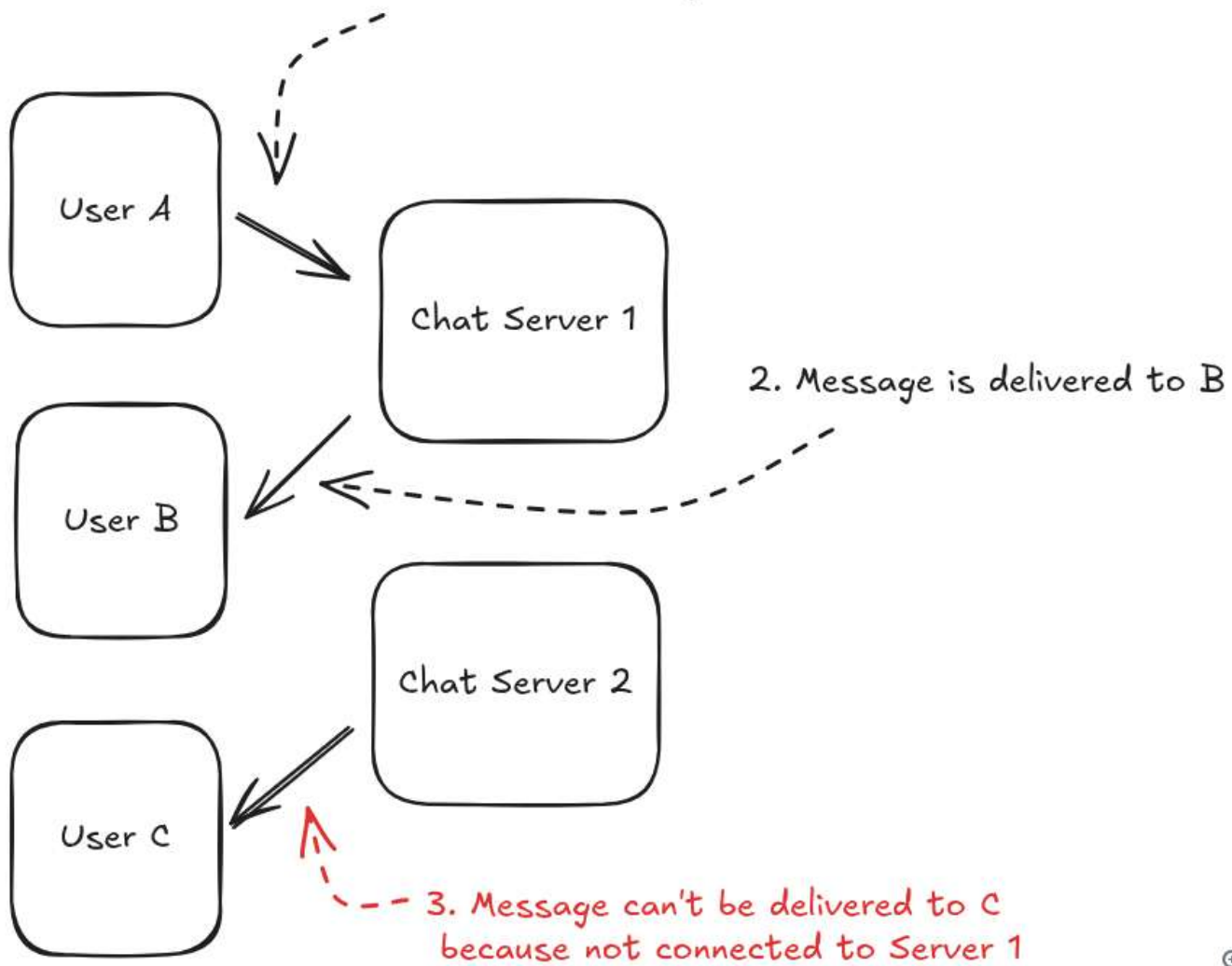
If we have 1b users, we might expect 200m of them to be connected at any one time. Whatsapp famously served 1-2m users per host, but this will require us to have hundreds of chat servers. That's a lot of simultaneous connections (!).



Note that I've included some back-of-the-envelope calculations here. Your interviewer will likely expect them, but you'll get more mileage from your calculations by doing them just-in-time: when you need to figure out a scaling bottleneck.

Adding more chat servers also introduces some new problems: now the sending and receiving users might be connected to different hosts. If User A is trying to send a message to User B and C, but User B and C are connected to different Chat Servers, we're going to have a problem.

1. User A tries to send a message to B and C



Host Confusion

The issue is one of routing: we're going to need to route messages to the right Chat Servers in order to deliver them. We have a few options here which are discussed in greatest depth in the [Realtime Updates Deep Dive](#).

Bad Solution: Naively horizontally scale

Approach

The most naive (broken) solution is to put a load balancer in front of our Chat Servers and scale horizontally.

Challenges

This won't work. A given server might accept a message to be sent to a Chat, but we're no longer guaranteed it will have the connections to each of the clients who needs to receive it. We won't be able to deliver the events and messages!

Bad Solution: Keep a kafka topic per user

Approach

Many candidates instinctively reach for a queue or stream in order to solve the scaling problem. One example solution would be to create a Kafka topic for every user in the system. The idea here would be that we could keep our **Inbox** table as a Kafka topic. Then our Chat Servers will subscribe to the topic and deliver messages to the user.

Under this proposal, when a user connects to our Chat Server, they'd subscribe to the topic for their user ID. When we need to send a message to a given user we'd publish the message to the topic for that user. This message would be received by all the chat servers that have subscribed to that topic, and then the message could be passed on to the websocket connection for that user.

Challenges

This unfortunately doesn't work - Kafka is not built for billions of topics and carries significant overhead for each one (order of 50kb per topic).

There are potential fixes that you might conceive, like creating "super topics" which group together all of the users on a given Chat Server, but you'll quickly find yourself reinventing the good aspects for alternative solutions below with little of the benefit.

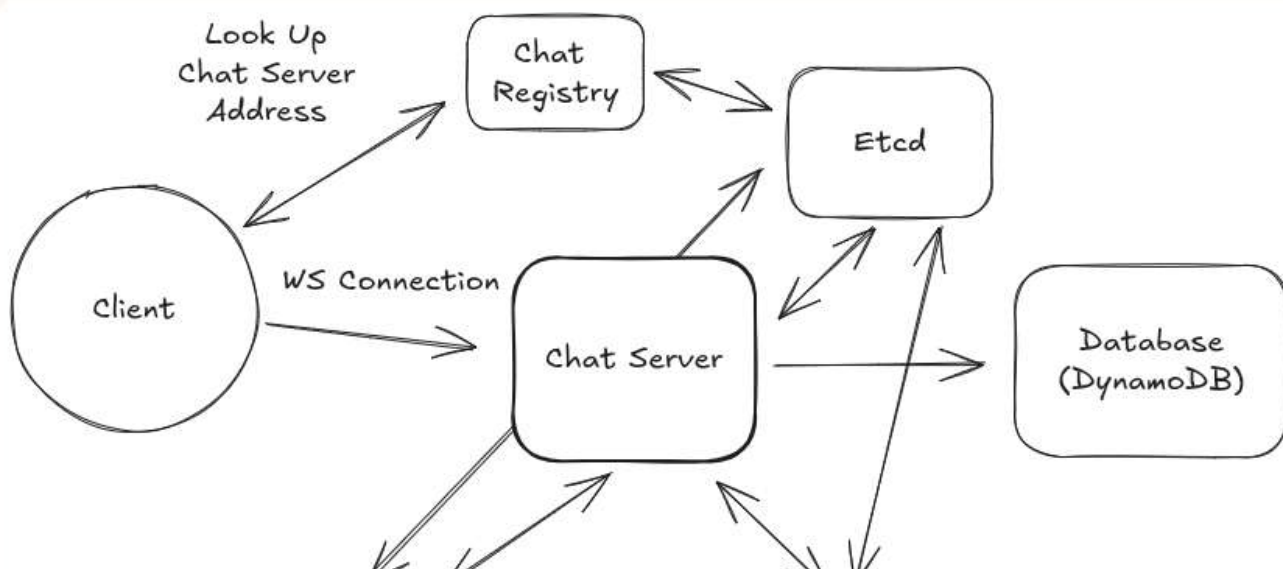
Good Solution: Consistent Hashing of Chat Servers

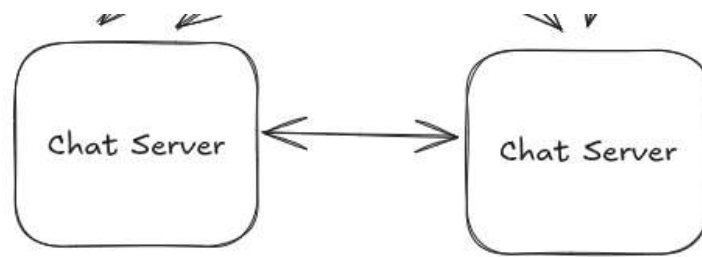
Approach

Another approach for us to use is to always assign users to a specific Chat Server based on their user ID. If we do this correctly, we'll always know which Chat Server is responsible for a given user so, when we need to send them messages, we can do so directly.

To do this we'll need to keep a central registry of how many Chat Servers we have, their addresses, and the which segments of a consistent hash space they own. We might use a service like **ZooKeeper** or Etcd to do this.

When a request comes in, we'll connect them to the Chat Server they are assigned to based on their user id. When a new event is created, Chat Servers will connect directly with the Chat Server that "owns" that user id, then call an API which delivers a notification the connected user (if they're connected).





Consistently Hashed Chat Servers

Challenges

Each Chat Server will need to maintain connections with each other Chat Server which will require that we keep our Chat Servers big in size and small in number.

Increasing the number of Chat Servers requires careful orchestration of dropping connections so that users reconnect to other servers without triggering a thundering herd (we need to be able to support users moving between servers). During scaling we need to ensure that events are sent to both servers to prevent dropping messages.

All of these are solvable problems but your interviewer will expect you to talk about them and how to pull it off.

Great Solution: Offload to Pub/Sub

Approach

The last approach is to use a purpose-built system for the bouncing messages between servers.

Redis Pub/Sub is a good example which uses a very lightweight hashmap of socket connections to allow you to ferry messages to arbitrary destinations. With Pub/Sub you can create a subscription for a given user ID and then publish messages to that subscription which are received "at most once" by the subscribers.

The difference here with Kafka is that Pub/Sub is not managing storage of messages. Pub/Sub uses a few bytes for each channel whereas disk-based Kafka requires many kilobytes per topic (in addition to the messages themselves).

On connection:

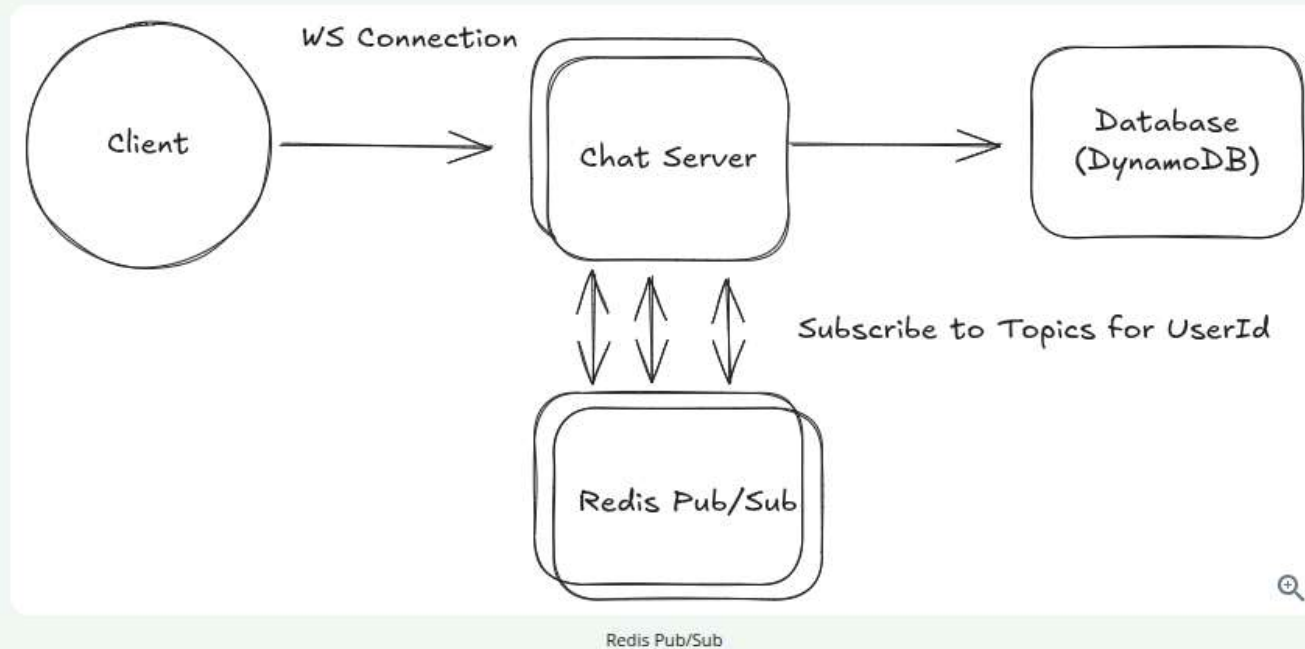
1. When users connect to our Chat Server, that server will connect to Pub/Sub to subscribe to the topic for that user ID.
2. Any messages received on that subscription are then forwarded on to the websocket connection for that user.

When a message needs to be sent:

1. We publish the message to the Pub/Sub topic for the relevant user ID.
2. The message is received by all subscribing Chat Servers.
3. Those Chat Servers then forward the message to the user's websocket connection.

Pub/Sub is "at most once" which means it doesn't guarantee delivery. If there's no subscribers listening, that pub/sub message is lost. But this is ok in our design, because we already manage the durability of our messages

with our `Inbox` table. If clients aren't connected, they'll receive the message when they connect later and retrieve all of the undelivered messages (we can also periodically poll for undelivered messages for transient failures in between).



Challenges

The Pub/Sub implementation introduces additional latency because we need to ferry messages via Redis. This is small (single-digit milliseconds) but it's still a cost.

We also have all-to-all relationship between Chat Servers and Redis cluster servers - we'll need to have connections between each Chat Server and each Redis server.

2) What do we do to handle multiple clients for a given user?

To this point we've assumed a user has a single device, but many users have multiple devices: a phone, a tablet, a desktop or laptop - maybe even a work computer. Imagine my phone had received the latest message but my laptop was off. When I wake it up, I want to make sure that all of the latest messages are delivered to my laptop so that it's in sync. We can no longer rely on the user-level "Inbox" table to keep track of delivery!

Having multiple clients/devices introduces some new problems:

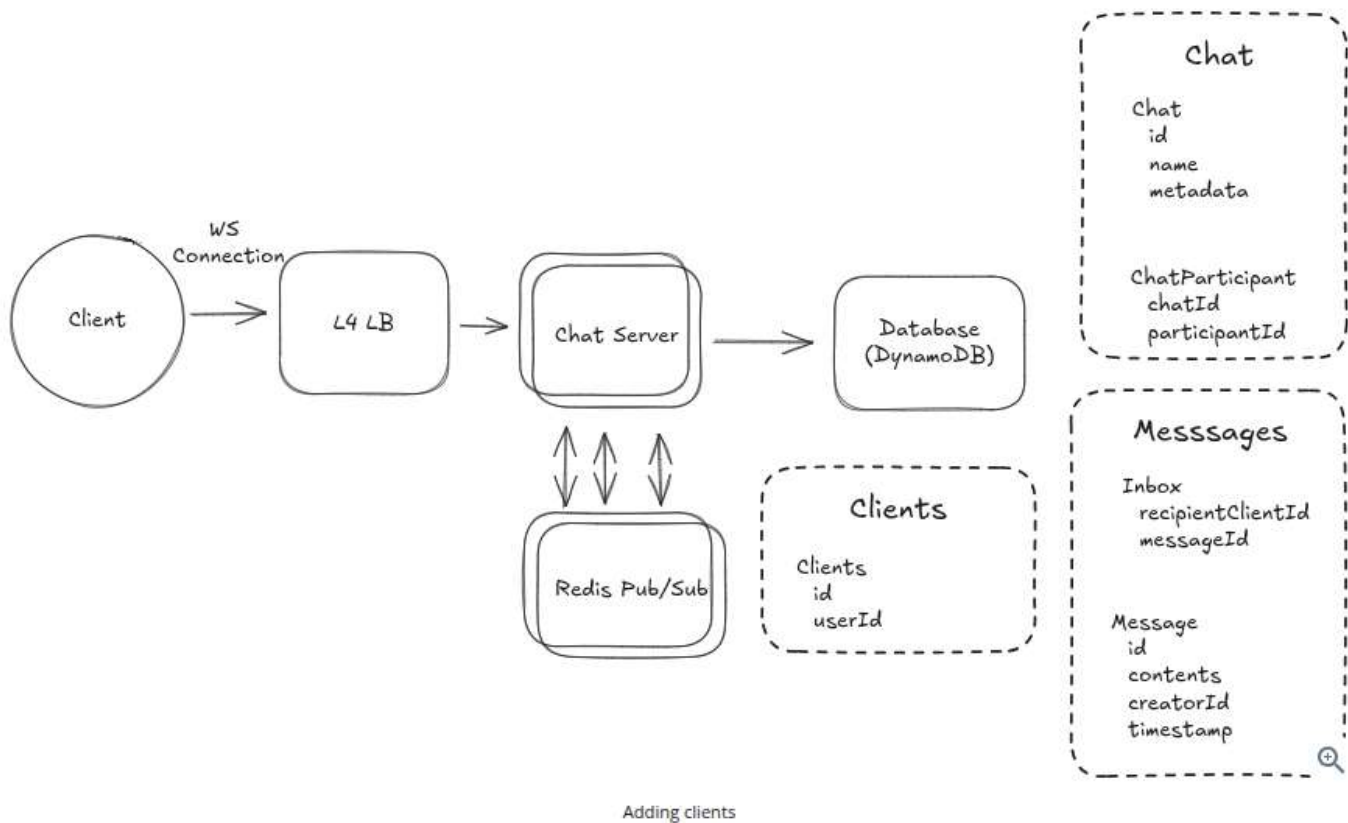
- First, we'll need to add a way for our design to resolve a user to 1 or more clients that may be active at any one time.
- Second, we need a way to deactivate clients so that we're not unnecessarily storing messages for a client which does not exist any longer.
- Lastly, we need to update our message delivery system so that it can handle multiple clients.

Let's see if we can account for this with minimal changes to our design.

- We'll need to create a new `clients` table to keep track of clients by user id.
- When we look up participants for a chat, we'll need to look up all of the clients for that user.
- We'll need to update our `Inbox` table to be per-client rather than per-user.
- When we send a message, we'll need to send it to all of the clients for that user.

- On the pub/sub side, nothing needs to change. Chat servers will continue to subscribe to a topic with the `userId`.

We'll probably want to introduce some limits (3 clients per account) to avoid blowing up our storage and throughput.



What is Expected at Each Level?

Ok, that was a lot. You may be thinking, "how much of that is actually required from me in an interview?" Let's break it down.

Mid-level

Breadth vs. Depth: A mid-level candidate will be mostly focused on breadth (80% vs 20%). You should be able to craft a high-level design that meets the functional requirements you've defined, but many of the components will be abstractions with which you only have surface-level familiarity.

Probing the Basics: Your interviewer will spend some time probing the basics to confirm that you know what each component in your system does. For example, if you use websockets, expect that they may ask you what it does and how they work (at a high level). In short, the interviewer is not taking anything for granted with respect to your knowledge.

Mixture of Driving and Taking the Backseat: You should drive the early stages of the interview in particular, but the interviewer doesn't expect that you are able to proactively recognize problems in your design with high precision. Because of this, it's reasonable that they will take over and drive the later stages of the interview while probing your design.

The Bar for Whatsapp: For this question, an E4 candidate will have clearly defined the API, landed on a high-level design that is functional and meets the requirements. Their scaling solution will have rough edges but they'll have

design that is functional and meets the requirements. Their scaling solution will have rough edges but they'll have some knowledge of its flaws.

Senior

Depth of Expertise: As a senior candidate, expectations shift towards more in-depth knowledge — about 60% breadth and 40% depth. This means you should be able to go into technical details in areas where you have hands-on experience. It's crucial that you demonstrate a deep understanding of key concepts and technologies relevant to the task at hand.

Advanced System Design: You should be familiar with advanced system design principles. For example, knowing about the consistent hashing for this problem is essential. You're also expected to understand the mechanics of long-running sockets. Your ability to navigate these advanced topics with confidence and clarity is key.

Articulating Architectural Decisions: You should be able to clearly articulate the pros and cons of different architectural choices, especially how they impact scalability, performance, and maintainability. You justify your decisions and explain the trade-offs involved in your design choices.

Problem-Solving and Proactivity: You should demonstrate strong problem-solving skills and a proactive approach. This includes anticipating potential challenges in your designs and suggesting improvements. You need to be adept at identifying and addressing bottlenecks, optimizing performance, and ensuring system reliability.

The Bar for Whatsapp: For this question, E5 candidates are expected to speed through the initial high level design so you can spend time discussing, in detail, scaling and robustness issues in the design. You should also be able to discuss the pros and cons of different architectural choices, especially how they impact scalability, performance, and maintainability.

Staff+

Emphasis on Depth: As a staff+ candidate, the expectation is a deep dive into the nuances of system design — I'm looking for about 40% breadth and 60% depth in your understanding. This level is all about demonstrating that, while you may not have solved this particular problem before, you have solved enough problems in the real world to be able to confidently design a solution backed by your experience.

You should know which technologies to use, not just in theory but in practice, and be able to draw from your past experiences to explain how they'd be applied to solve specific problems effectively. The interviewer knows you know the small stuff so you can breeze through that at a high level so you have time to get into what is interesting.

High Degree of Proactivity: At this level, an exceptional degree of proactivity is expected. You should be able to identify and solve issues independently, demonstrating a strong ability to recognize and address the core challenges in system design. This involves not just responding to problems as they arise but anticipating them and implementing preemptive solutions. Your interviewer should intervene only to focus, not to steer.

Practical Application of Technology: You should be well-versed in the practical application of various technologies. Your experience should guide the conversation, showing a clear understanding of how different tools and systems can be configured in real-world scenarios to meet specific requirements.

Complex Problem-Solving and Decision-Making: Your problem-solving skills should be top-notch. This means not only being able to tackle complex technical challenges but also making informed decisions that consider various factors such as scalability, performance, reliability, and maintenance.

Advanced System Design and Scalability: Your approach to system design should be advanced, focusing on scalability and reliability, especially under high load conditions. This includes a thorough understanding of distributed systems, load balancing, caching strategies, and other advanced concepts necessary for building robust, scalable systems.

The Bar for Whatsapp: For a staff+ candidate, expectations are high regarding depth and quality of solutions, particularly for the complex scenarios discussed earlier. Great candidates are going 2 or 3 levels deep to discuss failure modes, bottlenecks, and other issues with their design. There's ample discussion to be had around fault tolerance, database optimization, regionalization and cell-based architecture and more.

References

- [What Happens When You Make a Move in Lichess](#)



Test Your Knowledge

Take a quick 15 question quiz to test what you've learned.

Start Quiz

Mark as read ☐

[Next: Design a Rate Limiter →](#)

How would you rate the quality of this article?

☐ 1 ☐ 2 ☐ 3 ☐ 4 ☐ 5 ☐ 6 ☐ 7 ☐ 8 ☐ 9 ☐ 10

Poor Great

Add a comment...

B I </>

Comment

☐ Anonymous

Posting as leetcode

Q Search 566 comments

Sort By
Popular

K

kstarikov Top 1% • 7 months ago

we'll just use websockets although a simple TLS connection would do.

It's not clear how a 'simple TLS connection' would work here. I can imagine TLS being underneath the HTTPS connection that the client uses to send messages, but how will a server send message TO the client via just TLS?

We'll start with a simple service behind an L4 load balancer (to support Websockets!)

Why would we need an L4 LB to support websockets?

OK so I made it to the end of the article and I'm quite surprised by that this design covers only real-time delivery and doesn't talk about anything else. Like

- how clients get their messages when they reconnect.
- How the design manages message loss as Redis pub/sub is at most once delivery

- How the design manages shard masters' failures, because Redis pub/sub is not durable and if a message doesn't make it to the shard master, it won't ever be delivered to any of the shard subscribers

Also I'm wondering about how the write pipeline scales. Assuming 200 million of online users and each user writing one message per minute, we'll get 3.3M messages per second, that's 3.3M potential writes to the Message table every second. Assuming that chats have 50 participants on average, this is 165 million entries to be written to Inbox. How does our design

[Show More](#)

👍 62 [Reply](#)

F **ForwardIndigoCod967** • 3 months ago

+1, These are really good points, and I got these question in real interview

👍 5 [Reply](#)

R **RemainingTurquoiseGerbil100** Premium • 3 months ago

L7 load balancer terminates the connection and starts a new connection with the server. there are 2 tcp connections for L7 lb. client-to-lb and lb-to-server.

For L4 LB, there is only 1 TCP connection b/w client and server.

👍 5 [Reply](#)

J **Johnny Chang** Premium • 27 days ago

We don't really need the `Inbox` table to catch up.

We can add a cursor field `last_delivery_ts` in the `ChatParticipant` table. When the user is back online, the chat server:

1. select `last_delivery_ts` from `ChatParticipant` where `participant_id = user_id`
2. select `*` from `Messages` where `chat_id = :chat_id` and `timestamp > :last_delivery_ts`
3. deliver messages to the participant
4. update `ChatParticipant` set `last_delivery_ts = last_timestamp_from_message` where `chat_id = :chat_id` and `participant = :user_id`

👍 3 [Reply](#)

A **AmazingIndigoCaribou141** Premium • 23 days ago

Yeah but this is fan out on read

The design has the Messages sharded by chatid as the primary index (partition key, to be exact). When a user connects to a chat server, and is part of, say 1000 chats, they would have to do this expensive fan-out-on-read

ChatParticipants is sharded by clientid. When a message comes in, the write to CP table is at max 100 rows (as 100 participants) which is not a big problem (like Twitter celebrity fan-out-on-write to millions of partitions). On the flip side, the get-msgs-that-i-missed-offline is really efficient

👍 0 [Reply](#)



Haris Osmanagić Premium • 2 months ago

These are all good points, which I think can be relatively easily solved. But still, a huge thanks to the author for the huge amount of information that's given out for free.:

Assuming that chats have 50 participants on average, this is 165 million entries to be written to Inbox. How does our design handle that?

The design mentions that we don't have to write to the Inbox table if a message has been received by a user, i.e. we write to `Inbox` only if the send didn't work.

👍 0 [Reply](#)



Stan Premium • 2 months ago

The design mentions that we don't have to write to the Inbox table if a message has been received by a user, i.e. we write to Inbox only if the send didn't work.

I don't think it can work. If we use Inbox pattern we have to always write to it because Pub/Sub has no delivery guarantees.

 8 [Reply](#)

B

BackUp MyData Premium • 2 months ago

bro if you got the answer to your question , please compile list of such question and post, this will help us all for long. Thanks

 0 [Reply](#)

P

PreviousBlueHawk549 Premium • 13 days ago

I thought of having REST APIs for create chat, modify participants, attachment URLs, fetching Inbox content when user comes back online on a device While WebSocket is purposed only for sending messages, typing, presence, and delivery acks.

Rest APIs can be served over a separate service other than chat server.

Further I don't think Redis pub-sub alone will be able to entertain billions of channels named on userIds. What if sent notifications are dropped during transit between Redis and chat server ? We do have na inbox for the recipient device though and it can read on reconnects .

 0 [Reply](#)

N

NavalMaroonOtter533 • 1 year ago

Thank you SO much for these incredible answer keys. I have a question about handling when clients go offline. I see you mentioned:

Since our clients can go offline, we'll need to be able to "sync" state between the client and server. For convenience, let's just assume the server will buffer events sent while the client is offline and, when the client connects, it receives all unsent messages.

In the pub/sub solution, will this mean that messages will stay in a Redis event topic for that user until they come back online? If so won't that cause some sort of back pressure on Redis if a user accumulates a ton of unseen messages? Or maybe there's a separate message sync queue that the messages get moved to? I'm so curious!

 12 [Reply](#)

W

WorldwideBrownMollusk575 • 1 year ago

I can take a stab at this. Perhaps one thing would be that these messages are only via the redis pub/sub to online clients. For example if a client disconnects, that particular instance of the chat server no longer maintains `connectionId` of the respective client to send to.

When a client comes online, it will fetch all events from the DB with a state of `not_delivered` for example (perhaps via HTTP or we can use our websocket connection to send these on connect). Since we're offering message guarantees through our backend, the client can confirm that they've received this on the next online connection.

 12 [Reply](#)

R

RainyCyanHippopotamus256 ★ Top 10% • 1 year ago

How will we handle scenarios when the client is online but the persistent connection between the server and redis cluster breaks for a while and then reestablishes again? At this point , we will have undelivered messages and also live messages hitting through pub-sub simultaneously after the connection is reestablished. How will we consolidate both these type of messages while ensuring ordering guarantees?

 3 [Reply](#)



Stefan Mai Admin • 1 year ago

Stamp the messages on ingestion. Order them on the client. Good enough for chat.

👍 2 [Reply](#)

R

RainyCyanHippopotamus256 ★ Top 10% • 1 year ago

Yes, we can offload ordering on the client side by having the messages stamped with timestamp. But, still the low-level implementation or handling of the above scenario looks non-trivial atleast to me.

Will we always check for undelivered messages from the DB in the method which gets the redis pub-sub messages so that we always deliver the undelivered messages? This is sub-optimal. Also, polling at regular intervals won't work. I think it would be easier to implement this via redis streams which allows us to send ACKs and start from the previously left offset.

👍 1 [Reply](#)

C

ChemicalCopperIguana997 ★ Top 10% • 1 year ago

We can add some id on ingestion and when client gets real time messages from redis , it might find some ids missing based on last message received. It can query db for missed messages.
On every reconnection, client can have a db call

👍 1 [Reply](#)

Y

YOUNG M Premium • 1 year ago

can we checkpointing each client on the servers-side, and upon the new/reconnection, it will have a separate service read all the messages since that checkpoint and push to clients via the websocket?

👍 1 [Reply](#)

P

ProperLimePython690 Premium • 7 days ago

We can buffer the new messages from redis on chat server once the connection with redis is reestablished, at the same time we can query dynamo to get all the messages from a given timestamp. We can create a in memory priority queue on the chat servers for these messages and return the messages in order to the client.

👍 0 [Reply](#)

N

NavalMaroonOtter533 • 1 year ago

Ahhh ok that makes sense, thank you! I think I was mistakenly thinking of pub/sub as more of a queue than publisher subscriber model. Thanks!

👍 3 [Reply](#)



Stefan Mai Admin • 1 year ago

Thanks ZerothIndigoSalamander112 and great question ContinuedBeigeSwift112. The pub/sub solution is indeed "at most once" delivery - so we use it as a fast path but we need a secondary path to make sure messages are eventually delivered in the degenerate case. But nothing is stored on the pub/sub server besides a socket map.

👍 2 [Reply](#)

T

TechnicalScarletJellyfish241 • 1 year ago

Thanks for the reply Stefan, spent some time thinking and read this comment and it makes complete sense.

👍 0 [Reply](#)

M

MotionlessLavenderPelican114 • 1 year ago

Thanks for this comment! I also mistaken pub/sub as message queue , and had the same question. It makes total sense

to me now knowing that pub/sub does not retain messages

👍 0 [Reply](#)

D **DevelopedGreenShrew596** Premium • 3 months ago

I think this system design should improve in terms of better break down and better deep dive of concepts. To me it feels like a lot of it have been just skimmed through without real depth.

Maybe reduce the requirements and have 2/3 posts on different functional requirements

👍 11 [Reply](#)

E **EstablishedPurpleMarsupial260** Premium • 1 year ago

could you please go into more of a deep dive into how delivery of messages to offline clients is handled?

also on how to handle the last seen feature as mentioned below as well.

thank you!

👍 11 [Reply](#)

H **HeavyBrownAlbatross549** • 1 year ago

+1

👍 0 [Reply](#)

S **surajrider** ★ Top 5% • 10 months ago

+1

👍 0 [Reply](#)

he she Premium • 9 months ago

Not sure if my understanding is right here. Please correct me if it is wrong!

It seems the main difference between the BAD solution keep a KAFKA topic per user vs GREAT solution offload to Pub/Sub is using KAFKA or Redis??

Because it looks like both solutions are very similar which need to:

- 1 create a topic per users
- 2 subscribe to the topics of the users who are connected to the chat sever.
- 3 publish the message to the users' topics
- 4 send the message to users via the websockets.

Also, since a user (assuming no multiple clients) can connect to one chat server at a time, does it mean for each user id topic, it only can have one subscriber which is the chat server that the user is connected to? Although multiple chat servers may publish message there to that user.

am I missing something else?

👍 10 [Reply](#)

L **LogicalAmethystToad166** ★ Top 10% • 8 months ago

Topic per user is simply impossible. In production, we would most likely have a "live-messages" topic or something and it would have multiple partitions where users would map to these partitions via some sort of consistent hashing.. Topic per user in Kafka, with millions of active people at a time, is simply not possible, beyond Kafka's or many modern queue'ing technologies scope.. Correct me if I'm wrong here

👍 7 [Reply](#)

P **ProperLimePython690** Premium • 12 days ago

Kafka can only handle around 10k-20k topics. Redis pub sub can scale to million of topics as Redis treats each channel as

a lightweight pointers in RAM.

👍 0 [Reply](#)

[Show All Comments](#)

Schedule a mock interview

Meet with a FAANG senior+ engineer or manager and learn exactly what it takes to get the job.

[Schedule A Mock Interview](#)

Questions

[Meta SWE Interview Questions](#)
[Amazon SWE Interview Questions](#)
[Google SWE Interview Questions](#)
[OpenAI SWE Interview Questions](#)
[Engineering Manager \(EM\) Interview Questions](#)

Learn

[Learn System Design](#)
[Learn DSA](#)
[Learn Behavioral](#)
[Learn ML System Design](#)
[Learn Low Level Design](#)
[Guided Practice](#)

Links

[FAQ](#)
[Pricing](#)
[Gift Mock Interviews](#)
[Gift Premium](#)
[Become a Coach](#)
[Our Coaches](#)
[Hello Interview Premium](#)

Legal

[Terms and Conditions](#)
[Privacy Policy](#)

Contact

[About Us](#)
[Product Support](#)
7511 Greenwood Ave North
Unit #4238 Seattle
WA 98103